Connect6 Game Engine

(engine: `tbe`)

Tamara Besedina

November 2026

Abstract

The goal of this work is a Connect6 engine that is compatible with the Connect6GUI/Connect6Engine interface (Escalona Moncholí, 2026a), answers in real time, and uses evaluation-function weights, including weights learned with a machine-learning method (Escalona Moncholí, 2026b). The `tbe` engine was developed in five versions: v1 is a minimax search to a depth of two full moves; v2 adds alpha-beta pruning with move ordering and checks for immediate wins and defences; v3 adds an evaluation function with threats after Wu and Huang and connectivity after Liu; v4 adds quiescence search and forward pruning (forward pruning alone makes the search 2.7 times faster and compensates most of the cost of quiescence); v5 learns the weights by bootstrapping from search, which in the end showed that the original weights are better, so v5 is effectively the same as v4. Learning converges reproducibly from different starting points but does not make the engine stronger (45.8 % of the points against the hand-tuned weights). The final version passes all 12 test positions (win and defence in 1–3 moves) against Cloudict, survives 11 of its own moves on average against it from opening positions (v1: 6.5), and takes second place in the tournament after Cloudict.

1 Introduction

Connect6 is a two-player game of the k-in-a-row family. Black opens with a single stone; after that the players take turns placing *two* stones on empty intersections of a 19×19 board. The first player to form a line of six or more stones horizontally, vertically or diagonally wins. The 1–2–2 rule makes the game balanced: after their own move, each player has exactly one stone more on the board than the opponent (Wu and Huang, 2006).

The game is very hard to search: on an empty board a move is chosen from $\binom{361}{2} \approx 65\,000$ pairs of cells, and in the middle game from tens of thousands. Even a depth of two full moves is therefore impossible without a strong restriction of candidates, pruning, and a good position evaluation.

The work is organised as a chain of versions from v1 to v5, adding improvement. v4 added two techniques at once (quiescence and forward pruning); their contributions were separated in a dedicated experiment (Section 4.4). All experiments were run locally on a MacBook Air with Apple M1, 8 GB RAM, macOS 26.5.2: games against a human through the Connect-More GUI, tests, the tournament and training from the terminal. An example is shown on Fig. 1. Code: GitHub link.

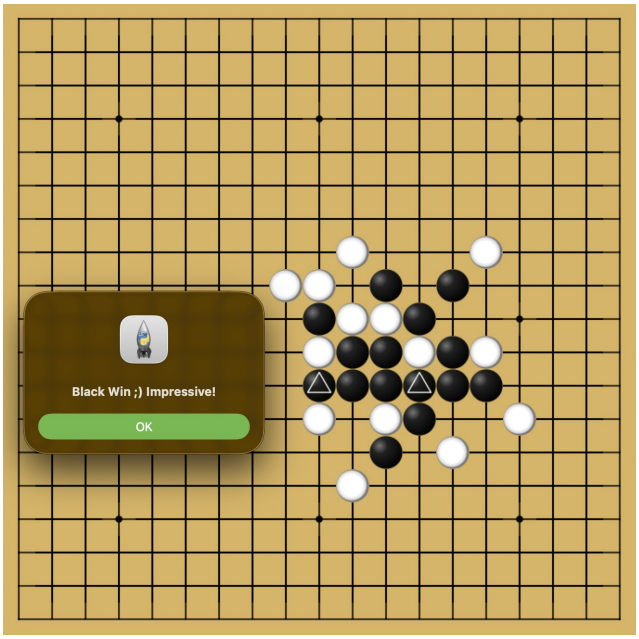


Figure 1: v2 (Black) against a human in the GUI.

2 State of the Art

2.1 k-in-a-Row Games and Connect6

Wu and Huang (2006) introduced the family of Connect(m, n, k, p, q) games and proposed Connect6 = Connect(19,19,6,2,1) as a fair game. The key concept we take from their work is the *threat*: a position in which the opponent has to spend a certain number of stones to avoid losing on the next move. If one stone is enough to defend, it is a single threat; if two are needed, a double threat; if two are not enough, the player with the threat wins. Since two stones are placed per move, a dead four (one end blocked) gives a single threat and an open four gives a double threat. The authors also stress that overlapping lines cannot be counted as independent threats and that the second stone of a move should be chosen after the first one has been placed.

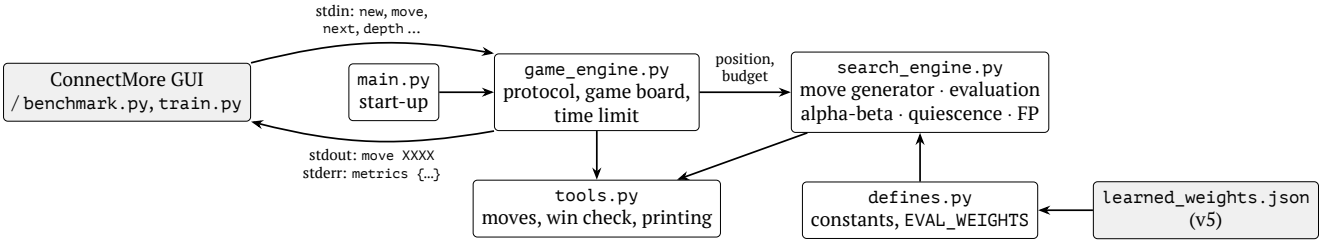


Figure 2: Architecture of the tbe engine and the flow of commands between the GUI, experiment scripts, and the engine. The modules are the same in all versions; mainly `search_engine.py` changes from version to version.

2.2 Game-Tree Search

The basic algorithm is depth-limited minimax with a static evaluation at the leaves (Russell and Norvig, 2020). Alpha-beta gives the same result but prunes branches that cannot affect the decision; with perfect move ordering the number of nodes drops to roughly the square root of minimax (Knuth and Moore, 1975), so move ordering is an essential part of alpha-beta. A fixed depth causes the *horizon effect*: a threat just beyond the search limit is not seen. Quiescence search continues the search at non-quiet leaves using tactical moves only (Russell and Norvig, 2020). Forward pruning discards in advance the moves that look weak according to a heuristic, trading accuracy for speed. The transposition tables, history heuristic and opening books suggested in the guide (Escalona Moncholí, 2026b) were not implemented.

2.3 Connect6-Specific Methods

The strongest Connect6 programs rely on threat-space search, introduced by Allis for Gomoku (Allis, 1994): only moves that create threats are considered, which makes it possible to find forced wins at a large depth. A special case is VCF (victory by continuous fours), a sequence of moves each of which creates a threat; this is how Cloudict finds its wins. Wu and Lin (2010) build on this idea a proof search over "relevance zones", the areas of the board where any defence must lie. Liu (2009) complements threat search with a defensive strategy and an evaluation of potential lines: the influence of a stone decreases with distance and does not pass through an opponent's stone. We use this idea in the connectivity feature.

2.4 Learning the Evaluation Function

An evaluation is often written as a linear combination of features (lines, threats) with weights. In NCTU6, a multiple Computer Olympiad champion in Connect6, the weights were first tuned by hand from games against strong players and then refined with TD(0) learning, which gave 57.95 % wins against the original version (Wu et al., 2012). The course guide suggests a genetic algorithm (Escalona Moncholí, 2026b): a population of weight sets whose fitness is the result of games between them. The alternative is learning from positions: TD-Leaf (Baxter et al., 2000) fits the evaluation to the search result from later positions, bootstrapping from game-tree search (Veness et al., 2009) fits it to the search value in the same position, and Texel tuning (Chess Programming Wiki, 2026) minimises a logistic error between the evaluation and a label of the position.

A genetic algorithm was not implemented because fitness in a GA is the result of games: evaluating a single weight set needs dozens of games, and a generation needs hundreds. Bootstrapping gives a label for *every* position rather than one per game, together with a differentiable error, so self-play give several thousand training examples and allow gradient-based optimisation.

2.5 Reference Opponent Cloudict

Cloudict (lang010, 2026) is an open-source Connect6 engine written in C++: alpha-beta to a given depth (2 in our setup) with VCF search. It serves as the reference opponent in all tests.

3 Development

3.1 Architecture

All versions share the same five-module structure given by the teacher's template (Fig. 2): `main.py` starts the engine, `game_engine.py` parses the protocol commands, `search_engine.py` contains the move generator, the evaluation and the search, `defines.py` holds constants and weights, and `tools.py` the board and the win check. The 19×19 board is stored in a 21×21 array with a border. Protocol answers go to stdout and search metrics (nodes, cut-offs, depth, time) to stderr: the GUI ignores them, while the experiment scripts read them.

3.2 Communication Protocol and Time Control

The engine supports all commands of the assignment (Table 1). As the assignment specifies, `depth N` sets a limit of N seconds per move rather than a search depth. Time is controlled as follows: the search is iterative (depth 1 first, then 2), the timer is checked at every node, and when the budget runs out the engine returns the best move of the last *fully* completed depth. A small part of the budget is kept for printing the answer.

Table 1: Commands of the Connect6GUI/Connect6Engine protocol.

Command	Action
`name`	answers `name tbe`
`new black` / `new white`	new game; the engine plays the given colour
`new xxx`, `black XXXX`, `white XXXX`	reset and rebuild an arbitrary position move by move (letters A–S)
`next`	compute a move for the side to move
`move XXXX`	place the opponent's move and immediately answer with its own (`move JKJL`; a one-stone move is `move JJ`)
`depth N`	time limit of N seconds per move (not the tree depth)
`print`, `help`, `exit`/`quit`	print the board, help, quit
`vcf`, `unvcf`	accepted and ignored, as the assignment requires

3.3 v1: Minimax Baseline

The first version meets the requirements of the first session of the guide. *End of game*: after each move the engine checks whether the placed stone completed six in a row in one of the four directions; a full board without a winner is a draw. *Move generator*: candidates are the empty cells within a radius of 3 of any stone; they are ranked by a simple local score, the 12 best cells are kept, and up to 24 pairs are built from them (a pair is scored by the sum of its cell scores). This generator is deliberately narrow: searching all pairs is impossible even at depth one. *Evaluation*: for every six-cell window without opponent stones the engine counts its own stones; the evaluation is the black sum minus the white sum, a win is $\pm 10^6$ and a draw is 0. *Search*: minimax to a depth of 2 full moves (a move and the reply).

3.4 v2: Alpha-Beta Pruning and Move Ordering

Alpha-beta. The method is built directly on the v1 minimax with the same terminal conditions and depth limit; on the same tree it returns the same move (checked by a test) but prunes branches when $\alpha \geq \beta$.

Ordering and forced moves. Alpha-beta is only efficient when strong moves are examined first (Knuth and Moore, 1975), so the generator became smarter. (1) If an immediate win exists, only winning pairs are proposed. (2) Otherwise, all opponent windows with 4–5 stones are collected and only pairs that block *all* of the opponent's ways to win on the next move are proposed; these cells enter the candidate list regardless of the limit of 12. If no two-stone defence exists, the search continues with ordinary moves so that the loss is evaluated honestly. (3) In a quiet position a pair is scored by the joint change of all affected windows: the two stones of one move can strengthen the same line, which the sum of individual cell scores does not see.

Weighted evaluation. The stone count in a window was replaced by a weight that grows by a factor of 4 per stone: 1, 4, 16, 64, 256 for a window with 1…5 own stones (justified in Section 3.5). Figure 1 shows an example game of v2 against a human.

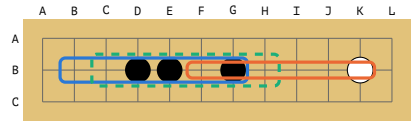
3.5 v3: Threat-Based Evaluation

The third version implements our own evaluation function based on the literature. Nine features are computed for each colour (Table 2, Fig. 3):

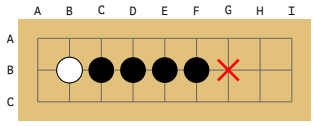
- `line_1`…`line_5`: the number of six-cell windows that contain 1…5 stones of this colour and no opponent stones (windows that can still become a six);
- `threat_1`, `threat_2`, `threat_3`: the threat after Wu and Huang (2006). For every window with 4–5 stones the engine stores the empty cells that would complete it, and then finds the minimum number of stones that block *all* these sets (taking intersections between directions into account). The flag `threat_1` means one stone is enough, `threat_2` that two are needed, `threat_3` that two are not enough;
- `connectivity`, an idea from Liu (2009): along each line, split by opponent stones into segments at least six cells long, every pair of own stones at distance $d \leq 5$ contributes $6 - d$. Close stones

Table 2: Features of the evaluation function and their hand-tuned weights used in v3, v4, and v5.

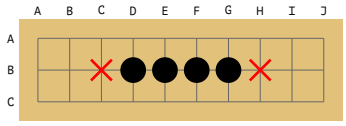
Feature	Weight	Meaning
`line_1`…`line_5`	1, 4, 16, 64, 256	six-cell windows with 1…5 own stones and no opponent stones; each stone multiplies the weight by 4
`threat_1`	600	all immediate wins can be blocked with one stone (dead four)
`threat_2`	3000	two stones are needed (open four)
`threat_3`	15000	two stones are not enough, so there is no defence
`connectivity`	2	sum of $(6 - d)$ over pairs of own stones on one unblocked line, $d \leq 5$



(a) six-cell windows: the blue and green ones contain 3 black stones each (`line_3`); the orange one contains a white stone and is not counted



(b) dead four: all candidate windows are blocked by one stone (×), which is `threat_1`



(c) open four: the windows are completed by the cell pairs, so the defence needs two stones, which is `threat_2`

Figure 3: Evaluation features.

are worth more, and the influence does not pass through an opponent stone.

The position evaluation is a weighted difference of the features:

$$E = \sum_i w_i \left(f_i^{\text{black}} - f_i^{\text{white}} \right), \tag{1}$$

where $E > 0$ means Black is better; a win is $\pm 10^6$ regardless of the weights.

Justification of the hand-tuned weights. The structure of the `line_k` features is standard for k-in-a-row games: for Gomoku, Szőts and Harmati (2018) count windows of length five without opponent stones and give a window with n stones the weight m^n (with $m = 10$ tuned through play). We use the same rule for windows of length six but with base 4. The base is smaller because two stones are placed per move in Connect6: a line can rise by two levels in a single move, and with $m = 10$ the evaluation would be determined by the single longest line, while shorter lines would no longer affect move choice. Learning in v5 indirectly supports this choice: the ratio `line_2`/`line_1` in the learned weights stayed close to 4 (Section 4.5). A linear combination of lines and threats with hand-tuned weights is the same scheme NCTU6 used before its TD learning (Wu et al., 2012). The threat weights were chosen from constraints: each threat level is 5 times stronger than the previous one; a single threat (600) outweighs several `line_4` fours because it forces the opponent to spend stones; and the upper bound of the whole feature sum stays far below the win score of 10^6. Connectivity is a weak positional feature, so its weight (2) is comparable to `line_1`–`line_2`.

Pair generator of v3. The v2 checks are kept. In a quiet position, following Wu and Huang, the first stone of a pair is temporarily placed on the board, and the candidates for the second stone are scored in the modified position. Pairs are sorted by outcome, then by threat strength (0/1/2/$\geq$ 3), by the number of live fours and fives, and by the evaluation (1); up to 24 pairs go into the search. The depth remained 2 full moves, but the evaluation became much more expensive (Section 4.3).

3.6 v4: Quiescence Search and Forward Pruning

At depth 2 the horizon effect is especially dangerous: an opponent win on the third move is not seen. v4 therefore adds two techniques from the fourth session of the guide; both can be switched off with the flags `--no-quiescence` and `--no-forward-pruning`, which later made it possible to measure them separately.

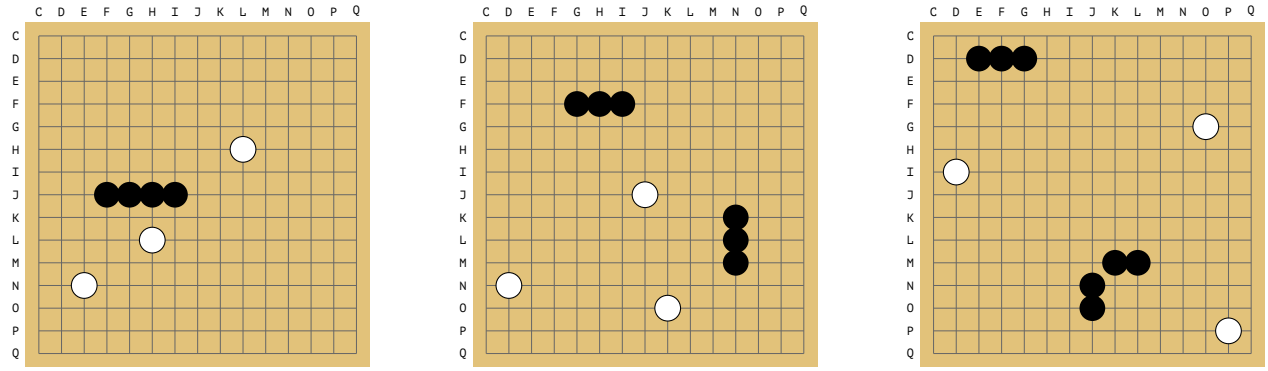
Quiescence search. At the depth limit, instead of the static evaluation, the engine runs a short tactical continuation: its own immediate wins first, otherwise the forced defences against an opponent win. In a quiet position the v3 evaluation is returned immediately and ordinary pairs are not generated. Under a threat the engine cannot "stand pat": if no pair blocks all of the opponent's wins, a losing score is returned. The extension depth is limited by `Q_MAX_DEPTH` and by the global timer.

Forward pruning. Ordinary pairs are first scored by the cheap change of the affected windows (the v2 score), and only the best 24 (`FP_PRESELECT`) go through the expensive v3 threat and connectivity analysis; after full ranking at most 8 quiet pairs remain (`FP_MAX_QUIET_MOVES`). Forcing pairs are never cut by this limit, and when either side has an immediate threat pruning is switched off completely and the v3 generator is used.

Tuning the quiescence depth. Initially two tactical moves were allowed (`Q_MAX_DEPTH = 2`). In a game against a human through the GUI (4 s limit) this caused a time-out in 4 of the 5 non-trivial moves: the search only completed depth 1 (Table 3, row "v4, QS = 2"). The problem was reproduced on two opening positions; with `Q_MAX_DEPTH = 1` depth 2 completed, and the chosen moves on four control positions were

Table 3: Search metrics in GUI games against a human (terminal logs, 4 s limit).

Version	moves	nodes	pruned	time, s (max)	depth	time-outs
v1	10	622	0	0.30 (0.33)	2.0	0
v2	6	97	246	0.13 (0.18)	2.0	0
v3	5	102	439	1.06 (1.48)	2.0	0
v4, QS = 2 (before tuning)	5	356	1797	3.37 (3.98)	1.2	4/5
v4, QS = 1 (final)	5	189	360	1.12 (3.24)	2.0	0



(a) win1: an open four, six on the next move

(b) win2: two open threes become two open fours, then six

(c) win3: an open three and two twos crossing at J M

Figure 4: Test position patterns (the attacker is Black; a fragment of the board is shown, filler stones on the edges are omitted). In the defend$k$ categories the engine plays White in the same positions; for White the patterns are rotated by $90°$.

unchanged. After tuning there were no time-outs in a similar game, and the average time per move dropped from 3.37 to 1.12 s.

The logs also confirm the cost of the v3 evaluation: on the first two moves v2 and v3 built the same tree (118 nodes, 506 pruned moves), but v3 spent 1.48 s against 0.18 s for v2, 8 times more, entirely because of the threat and connectivity features.

3.7 v5: Learning the Evaluation Weights

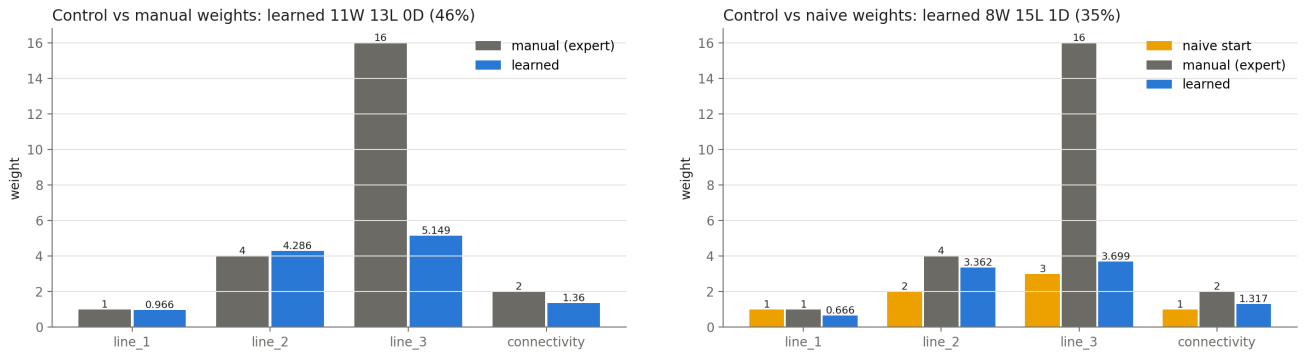
The fifth version uses the v4 search unchanged and adds weight learning (`train.py`). The learned weights are accepted only if they beat the hand-tuned ones both in error and in play.

Method is bootstrapping from search (Veness et al., 2009; Chess Programming Wiki, 2026). The static evaluation (1) learns to predict the value of a deeper search (the "teacher") in the same position. The teacher is a one-full-move search plus quiescence with the hand-tuned weights.

Data. Self-play with 6 processes, from random openings near the centre; during the first 6 moves, with probability $\varepsilon = 0.3$, the engine plays another of its four best moves instead of the best one, to diversify positions. Only *quiet* positions, where no side has 4–5 stones in a window, are labelled. These are exactly the leaves where the engine applies the static evaluation, while tactics are handled by the move generator and quiescence. Moreover, in tactical positions the teacher's value is close to $\pm 10^6$ and would dominate the error over all other positions. One run produced about 560 games and about 3 600 positions; 20 % of the *games* (not positions) were held out for validation, so that neighbouring positions of one game do not end up in both sets.

Error. Both evaluations are mapped to a "probability that Black wins" with the sigmoid $\sigma(E/K)$, where the scale $K = 108$ is the median of $|E_{\text{teacher}}|$; the error is the logistic loss (cross-entropy), as in Texel tuning (Chess Programming Wiki, 2026). The optimiser is Adam, with L2 regularisation towards the starting weights and early stopping on the validation error (Table 9).

Weights learned are `line_1`, `line_2`, `line_3` and `connectivity`. In quiet positions `line_4`, `line_5` and the threats are zero, there is no data for them, and they keep their hand-tuned values. The weights are bounded between 0.25 and 3.5–4 times the hand-tuned values and kept monotone, `line_1` < `line_2` < `line_3` < `line_4`. During training a side-to-move (tempo) term is added: the same position is better for the side that is to move. It is not needed in play: at a fixed depth the same side is to move in all leaves, so the term shifts all evaluations equally and does not change the choice of move.



(a) hand-tuned and learned weights

(b) naive, hand-tuned and learned weights

Figure 5: Final weights and the results of the control games (in the plot titles).

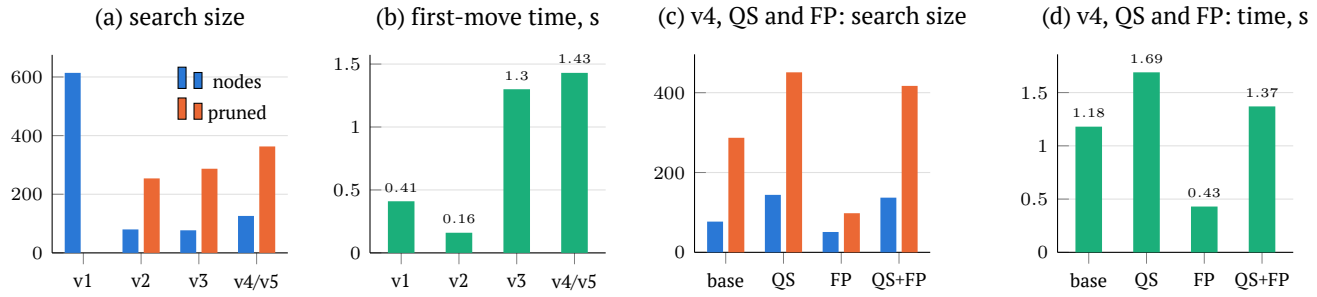


Figure 6: Engine metrics: (a, b) by version; (c, d) v4 with quiescence (QS) and forward pruning (FP) switched off and on. Blue bars are expanded nodes, orange bars are pruned moves.

Control and selection. The learned weights play against the starting weights: 12 new openings × 2 colours, 3 s per move, with a 40-move limit counted as a draw. The weights are accepted if the validation error is lower *and* they score more than 50 % in the control games; otherwise the starting weights are kept.

To check that the procedure finds reasonable weights on its own rather than just staying near the hand-tuned ones, a second experiment was run, starting from "naive" weights `line_1..3` = 1, 2, 3 (weight = number of stones) and `connectivity` = 1, with regularisation towards this naive start (not towards the hand-tuned weights, so as not to hint at the answer). The results are in Section 4.5.

4 Experimentation

4.1 Methodology

Experiments were run with the `benchmark.py` script. The opponent is Cloudict at depth 2 with VCF. Our engine's limit in the tests and the tournament is 3 s per move. *Test positions* were built following the fifth session of the guide (Fig. 4). For three patterns (a win in 1, 2 and 3 moves) the engine plays the attacker (categories win1–3), and in the same position with the roles reversed it plays the defender (defend1–3). Each pattern is used twice: for Black, and rotated by 90° for White. The stones needed to make the position legal are placed on the edge of the board, far from the pattern. A win$k$ test is passed if the engine wins within k of its own moves; a defend$k$ test if it does not lose within 4 of its own moves. The start category is an ordinary opening (as White after `JJ`, as Black after `JJ IKKK`): we count how many of its own moves (up to 15) the engine survives against Cloudict. Search metrics are taken from the *first* move, so all versions are compared on the same 14 positions.

4.2 Test Positions against Cloudict

The results are in Table 4.

(1) v1 fails win2, win3 and defend1: it has no explicit check for wins and defences, and its narrow generator (12 cells chosen by a simple score) does not always contain the required cells. From v2 on, all 12 tests are passed, thanks to the forced-move generator.

(2) All versions, including v1, pass defend2 and defend3: Cloudict does not manage to convert these threats within 4 moves even against a weak defender, so these categories hardly separate the versions.

(3) From the opening positions v2–v5 survive 11–12.5 of their own moves against Cloudict, while v1 survives only 6.5. The difference between v2, v3 and v4 (1–1.5 moves with two games per version) is within noise.

Table 4: Test positions against Cloudict: passed / played.

Version	win1	win2	win3	defend1	defend2	defend3	start
v1	2/2	0/2	0/2	0/2	2/2	2/2	6.5
v2	2/2	2/2	2/2	2/2	2/2	2/2	11.5
v3	2/2	2/2	2/2	2/2	2/2	2/2	12.5
v4/v5	2/2	2/2	2/2	2/2	2/2	2/2	11.0

A caveat: in the win3 positions the defender's filler stones sometimes lie on a line with its other stones, which gives the defender counter-play. This makes the test slightly harder than intended (this is how v1 lost win3), but it does not change the outcome for v2–v5.

4.3 Speed, Search Size and Memory

First-move metrics on the same 14 positions are given in Table 6 and Fig. 6(a, b).

(1) Going from v1 to v2 (alpha-beta and move ordering) reduces the number of nodes by a factor of 7.7, from 614 to 80, and the time by a factor of 2.5 at the same depth of 2.

(2) In v3 the number of nodes barely changes, but the time grows 8-fold, from 0.16 to 1.30 s: the threat and connectivity analysis at every node is expensive.

(3) In v4/v5 there are more nodes because of quiescence, and the average depth drops to 1.71: in some of the defend2 and defend3 positions the search hits the 3 s limit and only completes depth 1. In normal play through the GUI the limit is 4 s,

Table 5: v4 with QS and FP switched off and on, on the 14 test positions (3 s per move). Metrics are first-move averages; q-nodes are quiescence calls; FP-pruned are pairs removed by forward pruning.

Mode	tests	start	time, s	nodes	pruned	depth	q-nodes	FP-pruned	time-outs
no QS, no FP	12/12	12.5	1.18	77	287	2.00	0	0	0/14
QS only	12/12	10.5	1.69	144	451	1.64	131	0	5/14
FP only	12/12	13.5	**0.43**	51	98	2.00	0	159	0/14
QS + FP (v4)	12/12	11.0	1.37	137	417	1.71	127	137	4/14

and after tuning quiescence there were no time-outs (Table 3).

Table 6: First-move depth and memory averaged among 14 test positions.

Ver-n	depth	memory, MiB
v1	2.00	17.6
v2	2.00	18.0
v3	2.00	18.6
v4/v5	1.71	20.1

Memory. The peak resident size of the process (`/usr/bin/time -l`, two moves with a 3 s limit) is given in the last column of Table 6. All versions stay within 18–20 MiB, most of which is taken by the Python interpreter itself; the new techniques add about 2.5 MiB in total. Memory use is small by design: there are no transposition tables or caches, the recursion is at most three moves deep, and temporary stones are placed on and removed from a single board instead of copying it, so memory use is $O(\text{board} + \text{depth} \times \text{candidates})$ and does not grow with the number of nodes.

4.4 v4: Quiescence vs Forward Pruning

Both techniques were added to v4 at the same time, so their contributions were measured separately: the same set of 14 test positions was run for v4 in four modes, namely without either technique, with QS only, with FP only, and with both (the normal v4). The results are in Table 5 and Fig. 6(c, d).

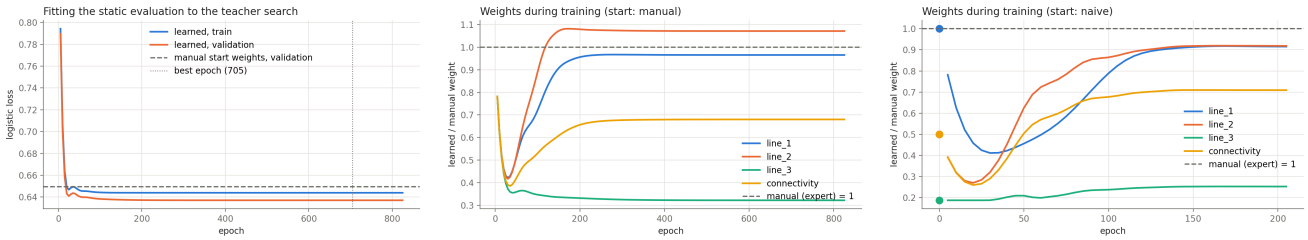
(1) This run is consistent with the main one: the mode without either technique gives exactly the same 77 nodes and 287 pruned moves as v3, and the QS+FP mode is close to the main v4 run (126 nodes, 1.43 s, depth 1.71). The small difference comes from the time limit: the moment when the search is interrupted varies slightly from run to run.

(2) Forward pruning is the more useful of the two techniques. It reduces the first-move time by a factor of 2.7, from 1.18 to 0.43 s, and the number of nodes by a third, without a single time-out or a single failed test.

(3) Quiescence, on the contrary, adds about 130 nodes and increases the time by a factor of 1.4. In 5 of the 14 positions the 3 s limit is not enough and the search only completes depth 1. When forward pruning is enabled as well, part of this cost is recovered: the time drops from 1.69 to 1.37 s, with 4 time-outs instead of 5.

(4) The test results do not separate the modes: all four pass 12 out of 12. Immediate wins and defences are already found by the move generator (since v2), so extending the search by one tactical move with quiescence adds nothing here. The start values (from 10.5 to 13.5 moves) come from two games per mode, and the difference of 1–3 moves is not significant.

In summary, forward pruning is the only one of the two techniques that speeds the search up; in v4 it compensates most of the cost of quiescence, so v4 is about as fast as v3 (1.37–1.43 s against 1.30 s). The benefit of quiescence is not visible on our tests, while its cost under a 3 s limit is noticeable. The final QS+FP configuration was not changed, and it is the one that played in the tournament.



(a) loss, start from hand-tuned weights **(b)** weights, start from hand-tuned **(c)** weights, start from naive (dots)

Figure 7: Training progress. (a) Logistic loss on train and validation when starting from the hand-tuned weights; the dashed line is the validation loss of the hand-tuned weights (0.6496), the best epoch is 705 (0.6371). (b, c) Weights as fractions of the hand-tuned values. The final weights are taken at the best epoch (705 and 85); the curves continue until early stopping. From both starts `line_3` ends up in the range 3.7–5.2, coming from above (from 16) and from below (from 3).

4.5 Weight Learning

The results of the two training runs, from the hand-tuned and from the naive weights, are given in Table 10; training progress and the final weights are shown in Figs. 7 and 5.

(1) The procedure works and reaches the same region of weights from different starting points. From the hand-tuned weights `line_3` falls from 16 to 5.1, and from the naive weights it rises from 3 to 3.7; from the naive start `line_2` rises from 2 to 3.4, almost matching the hand-tuned value of 4. The ratio `line_2/line_1` stays close to 4 in both runs, which supports the chosen base of 4.

(2) At the same time, the loss function hardly distinguishes between weight sets: the hand-tuned and the naive weights give practically the same validation loss (0.650 and 0.648), and training improves it by only 1–2 %.

(3) A lower loss does not translate into stronger play. In the control games the learned weights scored below 50 % in both runs: 45.8 % against the hand-tuned weights and 35.4 % against the naive ones. The learned weights undervalue `line_3`, while threes matter in play: a three of the side to move becomes a five in a single move.

(4) The cause most likely lies in the design of the features. Each feature is a difference "black minus white" and does not tell whose three it is: a three of the side to move is almost a win, a three of the opponent is not. The linear model averages these cases and undervalues `line_3`. The tempo term does not fix this, since it is the same for all positions and does not depend on the features.

(5) In the run from the hand-tuned weights the automatic selection rejected the learned weights, which is the right decision: v5 plays with the hand-tuned weights and is identical to v4. In the naive run the selection kept the naive starting weights; this run does not affect the weights the engine actually uses.

4.6 Tournament

All versions and Cloudict played a round robin: 2 openings (`JJ KJKK` and `JJ HHLL`) $\times$ 2 colours, 3 s per move, a 30-move limit, 60 games in total. 16 games did not finish within 30 moves and were scored as draws (0.5 points each). The standings and head-to-head results are in Table 7.

(1) The ranking follows the order of the improvements: v1, then v2, v3, v4 = v5, and Cloudict.

(2) v3 lost its head-to-head match against v2, 1.5:2.5: the new evaluation did not add strength by itself but slowed the search down 8-fold. v4 did not lose a single game either to v2 (2 wins, 2 draws) or to v3 (1 win, 3 draws). Given Section 4.4, a reasonable explanation is the speed-up from forward pruning, which lets the expensive v3 evaluation fit into the time limit.

(3) All versions lose to Cloudict, but v2–v5 survive noticeably longer than v1 (11–12.5 own moves in the start tests against 6.5), and v2 and v3 each reached the 30-move limit once. Cloudict is stronger thanks to the speed of C++ and its VCF search many moves ahead, while our search covers two full moves.

Table 7: Tournament standings (a win scores 1, a draw or an unfinished game 0.5). v4 and v5 played as separate entrants with identical results.

Player	points	W	L	D	games
Cloudict	19.0	18	0	2	20
v4	11.0	8	6	6	20
v5	11.0	8	6	6	20
v3	9.0	5	7	8	20
v2	8.5	5	8	7	20
v1	1.5	0	17	3	20

Table 8: Tournament head-to-head results: points of the row player against the column player (out of 4 games).

	v1	v2	v3	v4/v5	Cloudict
v1	—	0.5	0.0	0.5	0.0
v2	3.5	—	2.5	1.0	0.5
v3	4.0	1.5	—	1.5	0.5
v4/v5	3.5	3.0	2.5	2.0*	0.0
Cloudict	4.0	3.5	3.5	4.0	—

* v4 against v5: 2:2, the games mirror each other (same engine).

5 Conclusions

We developed the Connect6 engine `tbe`, fully compatible with the course protocol, in five versions, and measured the contribution of each step.

(1) v1 (minimax, depth 2) became a working baseline: 614 nodes per move, failed tests win2, win3 and defend1, 6.5 moves against Cloudict.

(2) v2 (alpha-beta, move ordering, forced moves) gave the largest gain: 7.7 times fewer nodes, all 12 tests passed, 11.5 moves against Cloudict.

(3) v3 (threats after Wu and Huang, connectivity after Liu) made the evaluation more meaningful but 8 times more expensive; in the tournament it lost to v2 by 1.5:2.5.

(4) v4 (quiescence, forward pruning) is the best version: second place in the tournament (11 points out of 20) with no losses to v2 or v3. A separate experiment showed that the speed-up comes from forward pruning (a factor of 2.7), while quiescence under a 3 s limit mostly adds cost.

(5) v5 (weight learning): the procedure converges re-

producibly from different starting points but does not improve play (45.8 % of the points against the original weights), so the automatic selection kept the hand-tuned weights.

Main lessons. The move generator and move ordering matter more than a sophisticated evaluation: almost all of the tactical strength appeared already in v2. An expensive evaluation only pays off when the search has time to apply it, which in our case is made possible by forward pruning. The learning procedure works, but it is limited by the features: they do not take into account whose turn it is, so the loss function can hardly tell good weights from bad ones.

Future work. (1) Separate features for the side to move and for the opponent, the main condition for weight learning to have an effect. (2) A deeper teacher and iterative bootstrapping (retraining on positions played with the learned weights). (3) VCF or threat-space search, as in Cloudict. (4) A transposition table that exploits board symmetries, and an opening book. (5) A tournament of v4 with forward pruning only, since the engine may be stronger without quiescence.

References

L. Victor Allis. *Searching for Solutions in Games and Artificial Intelligence*. PhD thesis, University of Limburg, Maastricht, 1994.

Jonathan Baxter, Andrew Tridgell, and Lex Weaver. Learning to play chess using temporal differences. *Machine Learning*, 40(3):243–263, 2000.

Chess Programming Wiki. Texel's tuning method. https://www.chessprogramming.org/Texel's_Tuning_Method, 2026. Accessed November 2026.

Félix Escalona Moncholí. Connect6GUI and Connect6Engine. https://github.com/felixem/Connect6GUI, https://github.com/felixem/Connect6Engine, 2026a. Accessed November 2026.

Félix Escalona Moncholí. Practice 1. Connect 6 game engine: Statement and guide. Computer Science and Artificial Intelligence, University of Alicante, 2026b.

Donald E. Knuth and Ronald W. Moore. An analysis of alpha-beta pruning. *Artificial Intelligence*, 6(4): 293–326, 1975.

lang010. Cloudict: a Connect6 engine. https://github.com/lang010/cloudict, 2026. Accessed November 2026.

Yung-ching Liu. A defensive strategy combined with threat-space search for Connect6. Master's thesis, National Taiwan Normal University, 2009.

Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020.

János Szőts and István Harmati. Development of an incremental pattern extraction based Gomoku agent. *Periodica Polytechnica Electrical Engineering and Computer Science*, 62(4):155–164, 2018. doi: 10.3311/PPee.11952.

Joel Veness, David Silver, William Uther, and Alan Blair. Bootstrapping from game tree search. In *Advances in Neural Information Processing Systems 22 (NIPS 2009)*, pages 1937–1945, 2009.

I-Chen Wu and Dei-Yen Huang. A new family of k-in-a-row games. In *Advances in Computer Games (ACG 11)*, volume 4250 of *Lecture Notes in Computer Science*, pages 180–194. Springer, 2006.

I-Chen Wu and Ping-Hung Lin. Relevance-zone-oriented proof search for Connect6. *IEEE Transactions on Computational Intelligence and AI in Games*, 2(3):191–207, 2010.

I-Chen Wu, Hsin-Ti Tsai, Hung-Hsuan Lin, Yi-Shan Lin, Chieh-Min Chang, and Ping-Hung Lin. Temporal difference learning for Connect6. In *Advances in Computer Games (ACG 2011)*, volume 7168 of *Lecture Notes in Computer Science*, pages 121–133. Springer, 2012.

A How to Run

All commands are run from the `tbe_engine` folder.

- Engine: `python3 v5_final/main.py` (or any `vN_.../main.py`); in the ConnectMore GUI use *Load Black/White Engine* and select `v5_final/main.py`.
- v4 without optimisations: `python3 v4_search_optimisations/main.py --no-quiescence --no-forward-pruning`.
- Tests, metrics and tournament: `python3 -B benchmark/benchmark.py`; v4 with QS and FP switched off and on: `python3 -B benchmark/benchmark.py --ablation`. A compiled Cloudict is required at `tbe_gui/engines/cloudict_native` (built with `cloudict_native/build.py`).
- Weight learning: `python3 -B v5_final/train.py` (starts from the hand-tuned weights; the result is written to `learned_weights.json`) or `--init naive`; plots: `python3 -B v5_final/plot.py`.

B Additional Tables

Table 9: Parameters of weight learning in v5 (`train.py`).

Parameter	Value	Parameter	Value
Teacher	1 full move + quiescence, hand-tuned weights	Error	logistic, $\sigma(E/K)$, K = median $\lvert E_{\text{teacher}}\rvert$ (≈ 108)
Self-play	6 processes, 15 min	Optimiser	Adam, lr = 0.05
Diversity	$\varepsilon = 0.3$ in the first 6 moves (among the 4 best)	Regularisation	L2 = 0.001 towards the starting weights
Positions	quiet only (no 4–5 stones in a window)	Epochs	up to 1500, early stopping, patience 120
Split	80/20 by games	Constraints	between 0.25 and 3.5–4 times the hand-tuned values, monotone `line_k`
Learned weights	`line_1..3`, `connectivity` (+ tempo)	Control	12 openings $\times$ 2 colours, 3 s/move, 40 moves

Table 10: Weight-learning results for the two runs: from the hand-tuned weights and from the naive weights. Control: games of the learned weights against the starting weights (12 openings $\times$ 2 colours). The validation error is the logistic loss on the held-out 20 % of games.

	Start from hand-tuned weights		Start from naive weights	
	hand-tuned (start)	learned	naive (start)	learned
`line_1`	1	0.966	1	0.666
`line_2`	4	4.286	2	3.362
`line_3`	16	5.149	3	3.699
`connectivity`	2	1.360	1	1.317
Validation loss	0.6496	0.6371	0.6481	0.6428
Games / positions (train + val)	562 / 2958 + 717		551 / 2910 + 687	
Best epoch	705		85	
Control vs start (W–L–D)	11–13–0 (45.8 %)		8–15–1 (35.4 %)	
Selected weights	hand-tuned		naive (starting)	